NAModule2Assignment

October 14, 2026

1 Module 2 Assignment

[50]:
```
import re
import time
import pickle


from networkx.drawing.nx_pydot import graphviz_layout
from networkx.algorithms import community
from networkx.algorithms.community import modularity
import networkx as nx

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
#import seaborn as sns
%matplotlib inline
```

1.1 Important

Please **AVOID** using `community` and `modularity` as your variable names. These are imported as preserved names for networkx submodules. Changing their representations would result in autograder failures.

[51]:
```
# disable warnings
import warnings
warnings.filterwarnings('ignore')
```

1.2 Part 1: Wikipedia Network with Communities

1.3 Data description

In this assignment, we are going to analyze the community structure of a network. We will use a Wikipedia based Map of Science network for our exploration. In this network, each node represents a Wikipedia page in a domain of science, such as natural science or social science. An edge exists between two nodes if the cosine similarity of their page contents reaches a pre-defined threshold.

[52]:
```
G = nx.read_gml('assets/MapOfScience.gml', label='id')
```

Each node in the graph contains the following attributes: - “name:” the title of the article - “Class” the science domain - “WikipediaUrl:” the Wikipedia URL

Let’s look at some examples:

[53]:
```
list(G.nodes(data=True))[0:5]
```

[53]:
```
[(0,
  {'label': '0',
   'name': 'Accounting',
   'Class': 'Applied',
   'WikipediaUrl': 'https://en.wikipedia.org/wiki/Accounting'}),
 (1,
  {'label': '1',
   'name': 'Aerospace engineering',
   'Class': 'Applied',
   'WikipediaUrl': 'https://en.wikipedia.org/wiki/Aerospace_engineering'}),
 (2,
  {'label': '2',
   'name': 'Agricultural engineering',
   'Class': 'Applied',
   'WikipediaUrl': 'https://en.wikipedia.org/wiki/Agricultural_engineering'}),
 (3,
  {'label': '3',
   'name': 'Agricultural science',
   'Class': 'Applied',
   'WikipediaUrl': 'https://en.wikipedia.org/wiki/Agricultural_science'}),
 (4,
  {'label': '4',
   'name': 'Agronomy',
   'Class': 'Applied',
   'WikipediaUrl': 'https://en.wikipedia.org/wiki/Agronomy'})]
```

The edges contain the cosine similarity of the text of the two articles

[54]:
```
list(G.edges(data=True))[0:5]
```

[54]:
```
[(0, 21, {'CosineSimilarity': 0.369447477753246}),
 (0, 50, {'CosineSimilarity': 0.395432741205435}),
 (0, 70, {'CosineSimilarity': 0.388758063740006}),
 (0, 88, {'CosineSimilarity': 0.371542879166867}),
 (0, 516, {'CosineSimilarity': 0.365688238862527})]
```

Let’s extract the largest connected component from the above graph. In the following questions, we will focus our analysis on this sub-graph.

[55]:
```
# This line extracts the largest connected component of the original dataset

G = G.subgraph(max(nx.connected_components(G), key=len))
```

1.3.1 Task 1a.

How many nodes are there in this new network? Assign the answer to the variable `N`.

Hint: the `.nodes()` function returns a list of all the nodes.

[56]:
```
task_id = '1a'
```

[57]:
```
def task_1a_solution():
    N = len(G.nodes())  # stores the number of nodes in this graph

    # YOUR CODE HERE
    #raise NotImplementedError()

    return N
```

[58]:
```
# Use this cell to explore your solution.

task_1a_solution()
```

[58]:
```
677
```

[59]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_1a_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, int
), f"Task {task_id}: Your function should return an integer. "


#hidden tests for Question 1 are within this cell
```

```
Task 1a - AG tests
Task 1a - your answer:
677
```

1.3.2 Task 1b:

If we think of science 'classes' as communities, how many communities are there in the network, and what are the names of all the classes?

Assign `N` to the number of classes in `G`, and `set_of_communities` to a set containing the names of the classes.

Hint: remember each node has attributes such as ['name'] and ['Class'], the latter of which is relevant for this assignment.

[60]:
```
task_id = '1b'
```

[61]:
```
def task_1b_solution():
    set_of_communities = set(G.nodes[node].get("Class") for node in G.nodes())␣
 ↪# set of unique community labels
    N = len(set_of_communities)           # number of communities in total

    # YOUR CODE HERE
    #raise NotImplementedError()

    return N, set_of_communities
```

[62]:
```
# Use this cell to explore your solution.

task_1b_solution()
```

[62]:
```
(4, {'Applied', 'Formal', 'Natural', 'Social'})
```

[63]:
```
print(f"Task {task_id} - AG tests")
stu_N, stu_set_of_communities = task_1b_solution()

print(f"Task {task_id} - your answer:\n{stu_N, stu_set_of_communities}")

assert isinstance(
    stu_N, int
), f"Task {task_id}: N should be an integer. "

assert isinstance(
    stu_set_of_communities, set
), f"Task {task_id}: set_of_communities should be a set. "


#hidden tests for Question 2 are within this cell
```

```
Task 1b - AG tests
Task 1b - your answer:
(4, {'Applied', 'Formal', 'Social', 'Natural'})
```

1.4 Part 2: Measures of Partition Quality

We discussed 5 ways to measure the quality of a partition: modularity, coverage, performance, separability, and density.

Modularity, coverage, and performance can be measured using `networkx` functions in the `algorithms.community` module. You can check all the functions provided by a module with the built-in `dir()` function:

```
from networkx.algorithms import community
dir(community)
>>> ...
 'modularity',
 'partition_quality',
```

Descriptions of the three measures are provided in the source code, (note that the second function returns both coverage and performance)

1. `networkx.algorithms.community.modularity(G, communities, weight='weight')`
2. `networkx.algorithms.community.partition_quality(G, partition)`

For separability and density, you will implement your own functions.

Since separability and density first apply a measure to each community and then takes the average over all communities, we provide a helper function `avg_measure(G, communities, measure)` that computes the average value for a given measure over all communities in a graph:

[64]:
```
def avg_measure(G, communities, measure):
    """
    Calculate the average value of a given measure across communities in a␣
 ↪graph.

    Args:
        G - nx graph object; a graph to operate upon.
        communities - list; a collection of sets of nodes that each comprise a␣
 ↪community.
        measure - function; calculates a measure for a community in a graph.

    Returns:
        (unnamed) - float; the calculated average measure.
    """
    sum_ = 0
    for comm in communities:
        sum_ += measure(G, comm)
    return sum_ / len(communities)
```

1.4.1 Task 2a:

Let's begin implementing the function to measure the separability for a single community. That is, measure the ratio of intra-community to inter-community edges and return it in `task_2a_solution`.

If there are 0 inter-community edges, the function should assume that the actual number is 1 and return the number of intra-community edges.

Hints:

- `G.edges(community)`, where `community` is a set of nodes, returns the edges that are incident to at least one node in `community`. That is, it returns the edges that have at least one endpoint in `community`.

- Remember that all edges are either inter- or intra- community edges.

[65]:
```
task_id = '2a'
```

[66]:
```
def task_2a_solution(G, community):
    """
    Calculate the separability of a community by finding the ratio of
    intra-community edges to inter-community edges.

    Args:
        G - nx graph object; a graph to operate upon.
        community - set; a collection of nodes that comprise a community.

    Returns:
        result - float; the separability in a given community.
    """
    result = None # assign separability to it and return
    edges = G.edges(community)
    num_intra_edges = 0
    num_inter_edges = 0

    for u, v in edges:
        if u in community and v in community:
            num_intra_edges += 1
        else: #elif u in community or v in community:
            num_inter_edges += 1

    # YOUR CODE HERE
    #raise NotImplementedError()
    #print(num_intra_edges, num_inter_edges)
    if num_inter_edges == 0:
        num_inter_edges = 1
    result = num_intra_edges / num_inter_edges

    return result
```

[67]:
```
# Use this cell to explore your solution.

task_2a_solution(G, [0,1])
```

[67]:
```
0.0
```

[68]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2a_solution(G, [0,1])

print(f"Task {task_id} - your answer:\n{stu_ans}")
```

```
assert isinstance(
    stu_ans, float
), f"Task {task_id}: Your function should return a float. "


#hidden tests for Question 4 are within this cell
```

```
Task 2a - AG tests
Task 2a - your answer:
0.0
```

Now we can simply use `avg_measure(G, communities, separability_one_community)` to measure the separability of a partition.

1.4.2 Task 2b:

Let's now implement the function to measure the density of a single community in `task_2b_solution`. That is, the fraction of intra-community edges out of all possible edges.

Hint: The equation to calculate density is

2 * (no. of intra-community edges) / (no. of community edges * (no. of community edges - 1))

[69]:
```
task_id = '2b'
```

[84]:
```
def task_2b_solution(G, community):
    """
    Calculate the density of a community by finding the fraction of
    intra-community edges out of all possible edges.

    Args:
        G - nx graph object; a graph to operate upon.
        community - set; a collection of nodes that comprise a community.

    Returns:
        result - float; the density of a given community.
    """

    result = None # assign density to it and return
    edges = G.edges(community)
    num_intra_edges = 0

    if len(community) == 1:  # If the community has only one node, just return␣
↪1
        return 1

    for u, v in edges:
        if u in community and v in community:
```

```
            num_intra_edges  += 1

    # YOUR CODE HERE
    #raise NotImplementedError()
    n = len(community)
    result = 2*num_intra_edges / (n*(n-1))

    return result
```

[85]:
```
# Use this cell to explore your solution.

task_2b_solution(G, [0,1])
```

[85]:
```
0.0
```

[72]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2b_solution(G, [0,1])

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, float
), f"Task {task_id}: Your function should return a float. "

#hidden tests for Question 5 are within this cell
```

```
Task 2b - AG tests
Task 2b - your answer:
0.0
```

Now we can simply use `avg_measure(G, communities, density_one_community)` to measure the density of a partition.

1.5 Part 3: Community Detection Algorithms

Now let's apply community detection algorithms to the Wikipedia graph. For this part, we will ignore the science domains since we are trying to find communities purely based on the structure of the network.

We will begin with the Girvan-Newman algorithm and pick the partition that results in the largest modularity, as described in the lecture. Since computing this partition takes a long time, we have precomputed the communities in the notebook `Girvan-Newman.ipynb` (stored in the resources folder) and exported the partition to a pickle file. The pickle file is stored as `assets/answer/max_mod_community`. Note that you do not need to run the notebook `Girvan-Newman.ipynb` since we have already done it for you. We only provide it to you as a reference.

Here, we will load the partition from `assets/answer/max_mod_community`

[73]:
```
max_mod_community = None

with open("assets/answer/max_mod_community", 'rb') as f:
    max_mod_community = pickle.load(f)
```

[74]:
```
print(max_mod_community)
```

```
({0, 1, 517, 519, 526, 19, 21, 533, 535, 24, 538, 27, 540, 29, 542, 31, 32, 543,
35, 37, 549, 554, 43, 42, 560, 561, 50, 562, 48, 565, 51, 55, 564, 569, 570, 59,
572, 575, 576, 65, 578, 69, 70, 583, 585, 587, 75, 77, 590, 591, 76, 80, 593,
82, 596, 83, 595, 599, 88, 89, 603, 604, 605, 96, 613, 614, 615, 616, 617, 618,
619, 620, 622, 111, 114, 627, 116, 626, 632, 634, 635, 637, 638, 127, 129, 641,
642, 133, 645, 647, 649, 650, 137, 140, 141, 654, 655, 656, 142, 143, 659, 148,
652, 145, 666, 155, 670, 158, 159, 674, 167, 177, 183, 187, 139, 188, 190, 191,
192, 193, 195, 202, 205, 217, 219, 226, 228, 234, 237, 247, 255, 257, 267, 268,
271, 273, 274, 275, 284, 302, 313, 365, 382, 426, 427, 453, 454, 456, 461, 462,
464, 467, 468, 470, 478, 480, 481, 482, 484, 489, 490, 492, 493, 495, 497, 498,
499, 500, 505, 506}, {2, 7, 9, 10, 521, 12, 16, 18, 26, 547, 36, 553, 556, 557,
580, 588, 84, 606, 609, 611, 105, 109, 625, 628, 122, 128, 130, 646, 653, 660,
668, 157, 672, 163, 204, 212, 233, 252, 272, 285, 287, 288, 289, 291, 301, 308,
312, 316, 318, 321, 325, 340, 342, 344, 350, 352, 354, 355, 363, 364, 370, 371,
372, 373, 374, 381, 388, 392, 399, 407, 421, 431, 434, 435, 442, 457, 458, 460,
472, 475, 485, 491, 494, 496, 503}, {417, 66, 3, 4, 41, 13, 415}, {5, 6, 11, 46,
52, 28}, {8, 15, 25, 34, 61, 86, 92, 93, 94, 95, 98, 99, 100, 101, 102, 103,
104, 107, 110, 112, 113, 115, 117, 119, 120, 123, 124, 125, 126, 131, 132, 134,
135, 136, 138, 144, 146, 151, 152, 153, 154, 156, 160, 161, 166, 169, 170, 171,
172, 173, 174, 175, 176, 179, 180, 181, 182, 184, 185, 186, 189, 194, 196, 197,
198, 199, 200, 201, 203, 206, 207, 208, 211, 214, 215, 216, 220, 221, 222, 223,
224, 227, 229, 230, 231, 232, 235, 236, 238, 239, 240, 242, 244, 245, 250, 253,
254, 262, 264, 276, 278, 279, 280, 281, 282, 283, 305, 326, 327, 386, 389, 398,
445, 488}, {14, 17, 20, 23, 54, 68, 79, 87, 209, 243, 258, 286, 290, 292, 293,
294, 295, 296, 297, 300, 303, 306, 309, 310, 311, 315, 317, 319, 320, 323, 328,
330, 331, 332, 335, 337, 338, 341, 351, 353, 357, 368, 369, 375, 377, 378, 379,
380, 383, 384, 385, 390, 391, 394, 395, 396, 397, 400, 404, 406, 409, 411, 412,
413, 416, 418, 419, 420, 428, 429, 430, 436, 437, 438, 439, 440, 441, 443, 444,
448, 449, 450}, {33, 67, 71, 72, 45, 81, 49, 405, 22, 62, 91, 30}, {38, 518, 40,
552, 522, 329, 73, 523, 47, 85, 662, 663, 664, 541, 510}, {610, 324, 548, 39,
360, 359, 298, 299, 366, 623, 432, 402, 403, 343, 476}, {546, 44, 358}, {60,
53}, {56, 57, 58}, {64, 333, 63}, {408, 74, 451, 90}, {393, 414, 78}, {256, 259,
260, 261, 263, 265, 266, 149, 277, 669, 164, 165, 592, 210, 345, 218, 97, 249,
361, 106, 108, 241, 246, 248, 633, 636}, {356, 425, 269, 118, 376}, {512, 513,
514, 515, 516, 643, 520, 511, 657, 531, 147, 534, 536, 539, 544, 162, 676, 555,
178, 563, 566, 568, 573, 577, 584, 586, 589, 336, 465, 594, 597, 600, 601, 602,
487, 624, 502, 121, 639}, {529, 658, 661, 150, 665, 551, 558, 579, 581, 582,
455, 459, 598, 477, 607, 479, 486, 621, 629, 507, 508}, {168, 501, 463}, {213},
{225}, {251}, {387, 270, 410, 422, 423, 304, 314, 574, 446, 447, 322, 452, 334,
339, 469, 347, 348, 349, 367}, {673, 471, 483, 644, 567, 648, 559, 307, 631,
```

```
504, 571}, {433, 346, 530, 532}, {362}, {424, 401}, {466}, {473, 651}, {537,
474, 545}, {608, 675, 612, 630, 667, 509}, {528, 524, 525, 527}, {550}, {640,
671})
```

1.5.1 Task 3a:

What is the modularity, coverage, performance, density, and separability of the partition?

[75]:
```
task_id = '3a'
```

[90]:
```
def task_3a_solution(Graph):

    mod = modularity(Graph, max_mod_community)    # modularity
    cov, perf = nx.algorithms.community.partition_quality(Graph,␣
 ↪max_mod_community)    # coverage # performance
    den = avg_measure(Graph, max_mod_community, task_2b_solution) # density
    sep = avg_measure(Graph, max_mod_community, task_2a_solution) # separability

    # YOUR CODE HERE
    #raise NotImplementedError()

    return mod, cov, perf, sep, den
```

[91]:
```
# Use this cell to explore your solution.

task_3a_solution(G)
```

[91]:
```
(0.5806872261399931,
 0.8152524167561761,
 0.8887058288830815,
 1.079485235171277,
 0.6542182538818658)
```

[92]:
```
print(f"Task {task_id} - AG tests")
stu_mod, stu_cov, stu_perf, stu_sep, stu_den = task_3a_solution(G)

print(f"Task {task_id} - your answer:\n{stu_mod, stu_cov, stu_perf, stu_sep,␣
 ↪stu_den }")

assert all(
    isinstance(num, float) for num in [stu_mod, stu_cov, stu_perf, stu_sep,␣
 ↪stu_den]
), (
    f"Task {task_id}: Your returned tuple should be a tuple of floats. "
)

#hidden tests for Question 8 are within this cell
```

```
Task 3b - AG tests
Task 3b - your answer:
(0.5806872261399931, 0.8152524167561761, 0.8887058288830815, 1.079485235171277,
0.6542182538818658)
```

1.5.2 Task 3b:

Find a partition of the network with the label propagation algorithm and compute the number of communities in the partition and its modularity, coverage, performance, density, and separability.

[79]:
```
task_id = '3b'
```

[93]:
```
def task_3b_solution(Graph):
    lp_communities = None # The lp partition. See Tutorial for details.
    num_community = None # number of communities in the partition
    mod = None   # modularity
    cov = None   # coverage
    perf = None  # performance
    den = None # density
    sep = None # separability

    # YOUR CODE HERE
    #raise NotImplementedError()

    from networkx.algorithms.community import partition_quality

    # Find communities using Label Propagation
    lp_communities = list(nx.algorithms.community.label_propagation.
 ↪label_propagation_communities(Graph))

    # Number of communities
    num_community = len(lp_communities)

    # Modularity
    mod = modularity(Graph, lp_communities)

    # Coverage and performance
    cov, perf = nx.algorithms.community.partition_quality(Graph, lp_communities)

    # Separability: fraction of edges incident to each community
    # that remain within that community

    den = avg_measure(Graph, lp_communities, task_2b_solution) # density
    sep = avg_measure(Graph, lp_communities, task_2a_solution) # separability

    return num_community, mod, cov, perf, den, sep, lp_communities
```

[94]:
```
# Use this cell to explore your solution.

task_3b_solution(G)
```

[94]:
```
(28,
 0.5841880852733243,
 0.8088077336197637,
 0.8578876526268868,
 0.6189993234063278,
 1.285532867611396,
 [{0,
   1,
   6,
   11,
   19,
   21,
   24,
   27,
   28,
   29,
   31,
   32,
   35,
   37,
   42,
   43,
   46,
   47,
   48,
   50,
   51,
   53,
   55,
   56,
   57,
   58,
   59,
   60,
   65,
   69,
   70,
   73,
   75,
   76,
   77,
   78,
   80,
```

82,
83,
85,
88,
89,
96,
111,
112,
114,
116,
118,
127,
129,
133,
137,
139,
140,
141,
142,
143,
145,
148,
154,
155,
158,
159,
167,
168,
173,
177,
183,
187,
188,
190,
191,
192,
193,
195,
202,
204,
205,
213,
217,
219,
225,
226,
228,

234,
237,
247,
251,
252,
255,
257,
267,
268,
269,
271,
273,
274,
275,
283,
284,
302,
307,
313,
329,
333,
362,
365,
382,
393,
414,
426,
427,
453,
454,
456,
457,
459,
461,
462,
463,
464,
466,
467,
468,
470,
471,
475,
478,
480,
481,
482,

483,
484,
489,
490,
492,
493,
495,
496,
497,
498,
499,
500,
501,
504,
505,
506,
517,
518,
519,
523,
526,
533,
535,
538,
540,
541,
542,
543,
549,
550,
554,
559,
560,
561,
562,
564,
565,
569,
570,
571,
572,
575,
576,
578,
583,
585,
587,

590,
591,
592,
593,
595,
596,
599,
603,
604,
605,
613,
614,
615,
616,
617,
618,
619,
620,
622,
626,
627,
631,
632,
633,
634,
635,
636,
637,
638,
641,
642,
644,
645,
647,
648,
649,
650,
652,
654,
655,
656,
659,
663,
664,
666,
670,
673,

```
 674},
{2,
 7,
 9,
 10,
 16,
 18,
 26,
 36,
 84,
 109,
 122,
 128,
 130,
 138,
 157,
 163,
 233,
 272,
 285,
 287,
 288,
 289,
 291,
 301,
 308,
 316,
 318,
 321,
 325,
 340,
 342,
 344,
 350,
 352,
 354,
 355,
 363,
 364,
 370,
 372,
 373,
 374,
 381,
 388,
 392,
 399,
```

```
 407,
 421,
 431,
 434,
 435,
 442,
 458,
 460,
 472,
 485,
 491,
 494,
 503,
 521,
 547,
 553,
 556,
 557,
 567,
 588,
 606,
 609,
 611,
 625,
 628,
 646,
 653,
 660,
 668,
 672},
{3, 4, 415, 417},
{5, 52},
{8,
 68,
 92,
 93,
 94,
 95,
 98,
 99,
 100,
 102,
 107,
 115,
 123,
 126,
 151,
```

152,
153,
156,
160,
161,
166,
169,
170,
171,
174,
175,
176,
179,
180,
181,
184,
185,
186,
189,
196,
197,
198,
199,
200,
201,
206,
208,
209,
211,
214,
215,
216,
220,
221,
222,
229,
230,
231,
232,
235,
236,
239,
240,
243,
244,
245,
250,

```
 253,
 258,
 262,
 276,
 278,
 280,
 281,
 286,
 296,
 300,
 309,
 312,
 323,
 328,
 330,
 335,
 338,
 353,
 376,
 378,
 379,
 395,
 397,
 398,
 404,
 418,
 420,
 429,
 437,
 440,
 448,
 450},
{12, 371, 580},
{13, 41, 66},
{14,
 20,
 23,
 54,
 63,
 64,
 79,
 290,
 293,
 297,
 303,
 306,
 310,
```

```
 311,
 315,
 317,
 319,
 320,
 331,
 337,
 341,
 351,
 356,
 357,
 368,
 369,
 375,
 377,
 380,
 383,
 384,
 385,
 390,
 391,
 394,
 396,
 400,
 406,
 409,
 411,
 413,
 419,
 425,
 428,
 430,
 436,
 438,
 439,
 441,
 443,
 444,
 449},
{15,
 25,
 34,
 61,
 86,
 101,
 103,
 104,
```

```
 110,
 113,
 117,
 119,
 120,
 124,
 125,
 131,
 132,
 134,
 135,
 136,
 144,
 146,
 172,
 182,
 194,
 203,
 207,
 223,
 224,
 227,
 238,
 242,
 254,
 264,
 279,
 282,
 305,
 326,
 327,
 386,
 389,
 445,
 488},
{17, 87, 292, 294, 295, 332, 403, 412, 416},
{22, 30, 33, 45, 49, 62, 67, 71, 72, 81, 91, 405},
{38, 40, 270, 304, 422, 469, 552, 574, 662},
{39, 298, 299, 324, 343, 359, 360, 366, 402, 432, 476, 548, 610, 623},
{44, 358, 546},
{74, 90, 408, 451},
{97,
 106,
 108,
 149,
 164,
 165,
```

```
 210,
 218,
 241,
 246,
 248,
 249,
 256,
 259,
 260,
 261,
 263,
 265,
 266,
 277,
 345,
 361,
 669},
{105, 212},
{121,
 147,
 162,
 178,
 336,
 465,
 487,
 502,
 510,
 511,
 512,
 513,
 514,
 515,
 516,
 520,
 522,
 531,
 534,
 536,
 539,
 544,
 555,
 563,
 566,
 568,
 573,
 577,
 584,
```

```
 586,
 589,
 594,
 597,
 600,
 601,
 602,
 624,
 639,
 640,
 643,
 657,
 671,
 676},
{150,
 455,
 477,
 479,
 486,
 507,
 508,
 529,
 551,
 558,
 579,
 581,
 582,
 598,
 607,
 621,
 629,
 658,
 661,
 665},
{314, 367},
{322, 348, 410, 446, 447, 452},
{334, 339},
{346, 347, 349, 387, 423, 433, 530, 532},
{401, 424},
{473, 651},
{474, 537, 545},
{509, 608, 612, 630, 667, 675},
{524, 525, 527, 528}])
```

[96]:
```
print(f"Task {task_id} - AG tests")
stu_num_community, stu_mod, stu_cov, stu_perf, stu_den, stu_sep,␣
 ↪stu_lp_communities = task_3b_solution(G)
```

```
print(f"Task {task_id} - your answer:\n{stu_num_community, stu_mod, stu_cov,␣
 ↪stu_perf, stu_den, stu_sep, stu_lp_communities}")

assert isinstance(
    stu_num_community, int
), f"Task {task_id}: num_community should be an int. "

assert all(
    isinstance(num, float) for num in [stu_mod, stu_cov, stu_perf, stu_den,␣
 ↪stu_sep]
), (
    f"Task {task_id}: mod, cov, perf, den, and sep should be floats. "
)

assert isinstance(
    stu_lp_communities, list
), f"Task {task_id}: lp_communities should be a list. "

#hidden tests for Question 9 are within this cell
```

```
Task 3b - AG tests
Task 3b - your answer:
(28, 0.5841880852733243, 0.8088077336197637, 0.8578876526268868,
0.6189993234063278, 1.285532867611396, [{0, 1, 517, 6, 518, 519, 11, 523, 526,
19, 21, 533, 535, 24, 538, 27, 28, 29, 540, 31, 32, 541, 542, 35, 543, 37, 549,
550, 42, 43, 554, 46, 47, 48, 559, 50, 51, 560, 53, 561, 55, 56, 57, 58, 59, 60,
564, 565, 569, 570, 65, 571, 572, 575, 69, 70, 576, 578, 73, 583, 75, 76, 77,
78, 585, 80, 587, 82, 83, 590, 85, 591, 592, 88, 89, 593, 595, 596, 599, 603,
604, 96, 605, 613, 614, 615, 616, 617, 618, 619, 620, 622, 111, 112, 114, 626,
116, 627, 118, 631, 632, 633, 634, 635, 636, 637, 638, 127, 129, 641, 642, 644,
133, 645, 647, 648, 137, 649, 139, 140, 141, 142, 143, 650, 145, 652, 654, 148,
655, 656, 659, 663, 664, 154, 155, 666, 158, 159, 670, 673, 674, 167, 168, 173,
177, 183, 187, 188, 190, 191, 192, 193, 195, 202, 204, 205, 213, 217, 219, 225,
226, 228, 234, 237, 247, 251, 252, 255, 257, 267, 268, 269, 562, 271, 273, 274,
275, 283, 284, 302, 307, 313, 329, 333, 362, 365, 382, 393, 414, 426, 427, 453,
454, 456, 457, 459, 461, 462, 463, 464, 466, 467, 468, 470, 471, 475, 478, 480,
481, 482, 483, 484, 489, 490, 492, 493, 495, 496, 497, 498, 499, 500, 501, 504,
505, 506}, {128, 2, 130, 388, 646, 7, 392, 9, 10, 138, 521, 653, 399, 16, 272,
18, 660, 407, 26, 668, 285, 157, 287, 288, 289, 672, 163, 36, 291, 421, 547,
553, 556, 301, 557, 431, 434, 435, 308, 567, 442, 316, 318, 321, 325, 458, 460,
588, 84, 340, 342, 344, 472, 350, 606, 352, 609, 354, 355, 611, 485, 233, 363,
364, 109, 491, 494, 625, 370, 372, 373, 374, 503, 628, 122, 381}, {417, 3, 4,
415}, {52, 5}, {8, 68, 92, 93, 94, 95, 98, 99, 100, 102, 107, 115, 123, 126,
151, 152, 153, 156, 160, 161, 166, 169, 170, 171, 174, 175, 176, 179, 180, 181,
184, 185, 186, 189, 196, 197, 198, 199, 200, 201, 206, 208, 209, 211, 214, 215,
216, 220, 221, 222, 229, 230, 231, 232, 235, 236, 239, 240, 243, 244, 245, 250,
```

```
253, 258, 262, 276, 278, 280, 281, 286, 296, 300, 309, 312, 323, 328, 330, 335,
338, 353, 376, 378, 379, 395, 397, 398, 404, 418, 420, 429, 437, 440, 448, 450},
{371, 12, 580}, {41, 66, 13}, {384, 385, 390, 391, 394, 396, 14, 400, 20, 406,
23, 409, 411, 413, 290, 419, 293, 297, 425, 428, 430, 303, 306, 436, 54, 310,
311, 438, 439, 315, 441, 317, 443, 319, 64, 320, 63, 444, 449, 331, 79, 337,
341, 351, 356, 357, 368, 369, 375, 377, 380, 383}, {386, 131, 132, 389, 134,
135, 136, 264, 15, 144, 146, 279, 25, 282, 34, 172, 305, 182, 61, 445, 194, 326,
327, 203, 207, 86, 223, 224, 227, 101, 103, 104, 488, 110, 238, 113, 242, 117,
119, 120, 124, 125, 254}, {416, 292, 294, 295, 332, 17, 403, 87, 412}, {33, 67,
71, 72, 45, 49, 81, 405, 22, 62, 91, 30}, {422, 38, 40, 552, 270, 304, 469, 662,
574}, {610, 324, 548, 39, 359, 360, 298, 299, 366, 623, 432, 402, 343, 476},
{546, 44, 358}, {408, 74, 451, 90}, {256, 259, 260, 261, 263, 265, 266, 149,
277, 669, 164, 165, 210, 345, 218, 97, 361, 106, 108, 241, 246, 248, 249}, {105,
212}, {512, 513, 514, 515, 516, 640, 643, 520, 522, 639, 657, 147, 531, 534,
536, 539, 671, 544, 162, 676, 555, 178, 563, 566, 568, 573, 577, 584, 586, 589,
336, 465, 594, 597, 600, 601, 602, 487, 624, 502, 121, 510, 511}, {529, 658,
661, 150, 665, 551, 558, 579, 581, 582, 455, 598, 477, 607, 479, 486, 621, 629,
507, 508}, {314, 367}, {322, 452, 410, 348, 446, 447}, {339, 334}, {387, 423,
433, 530, 532, 346, 347, 349}, {424, 401}, {473, 651}, {537, 474, 545}, {608,
675, 612, 630, 667, 509}, {528, 524, 525, 527}])
```

The Clauset-Newman-Moore greedy modularity maximization algorithm implements an Agglomerative Hierarchical Clustering procedure to find a partition with high modularity.

Note: the function `greedy_modularity_communities` returns a list of `Frozensets`, where each `Frozenset` is simply an immutable Python `set` object (i.e. the elements cannot be modified).

1.5.3 Task 3c:

Using Clauset-Newman-Moore greedy modularity maximization, find a partition of the network and compute the number of communities in the partition and its modularity, coverage, performance, density, and separability.

[ ]:
```
task_id = '3c'
```

[97]:
```
def task_3c_solution(Graph):
    gred_communities =  nx.algorithms.community.
 ↪greedy_modularity_communities(Graph)
    num_community = len(gred_communities) # number of communities in the␣
 ↪partition
    mod = modularity(Graph, gred_communities)
    cov, perf = nx.algorithms.community.partition_quality(Graph,␣
 ↪gred_communities)
    den = avg_measure(Graph, gred_communities, task_2b_solution) # density
    sep = avg_measure(Graph, gred_communities, task_2a_solution) # separability
    return num_community, mod, cov, perf, den, sep, gred_communities
```

[98]:
```
task_3c_solution(G)
```

```
[98]: (14,
       0.5503560299288301,
       0.8058922817247199,
       0.7750954874009073,
       0.6085197486447432,
       1.4808561418725692,
       [frozenset({0,
                   21,
                   38,
                   40,
                   42,
                   50,
                   56,
                   57,
                   58,
                   59,
                   65,
                   69,
                   70,
                   73,
                   74,
                   75,
                   77,
                   78,
                   80,
                   82,
                   90,
                   121,
                   127,
                   147,
                   148,
                   150,
                   162,
                   178,
                   202,
                   270,
                   273,
                   287,
                   302,
                   304,
                   307,
                   313,
                   322,
                   329,
                   336,
                   346,
                   347,
```

348,
349,
362,
382,
387,
393,
408,
410,
414,
422,
423,
426,
427,
433,
446,
447,
451,
452,
453,
454,
455,
456,
458,
459,
461,
462,
464,
465,
466,
467,
468,
469,
470,
471,
472,
475,
477,
478,
479,
480,
482,
483,
484,
486,
487,
489,
490,

492,
493,
495,
496,
497,
498,
499,
500,
502,
504,
505,
506,
507,
508,
510,
511,
512,
513,
514,
515,
516,
517,
518,
519,
520,
522,
523,
526,
529,
530,
531,
532,
533,
534,
535,
536,
539,
540,
541,
542,
543,
544,
549,
550,
551,
552,
554,

555,
558,
559,
561,
562,
563,
566,
567,
568,
569,
570,
571,
572,
573,
574,
575,
576,
577,
578,
579,
581,
582,
583,
584,
585,
586,
589,
590,
591,
593,
594,
595,
596,
597,
598,
599,
600,
601,
602,
603,
604,
605,
607,
613,
614,
615,
616,

```
            617,
            618,
            619,
            620,
            621,
            624,
            626,
            627,
            629,
            631,
            632,
            634,
            635,
            636,
            637,
            638,
            639,
            640,
            641,
            642,
            643,
            644,
            645,
            647,
            648,
            649,
            650,
            652,
            654,
            655,
            656,
            657,
            658,
            659,
            661,
            662,
            663,
            664,
            665,
            666,
            670,
            671,
            673,
            674,
            676}),
    frozenset({2,
            3,
```

```
4,
7,
8,
9,
10,
12,
13,
16,
17,
18,
23,
26,
36,
39,
41,
44,
54,
63,
64,
66,
68,
79,
84,
87,
99,
109,
122,
126,
128,
130,
138,
151,
153,
157,
173,
209,
233,
243,
253,
258,
272,
285,
286,
288,
289,
290,
291,
```

292,
293,
294,
295,
296,
297,
298,
299,
300,
301,
303,
306,
308,
309,
310,
311,
312,
315,
316,
317,
318,
319,
320,
321,
323,
324,
325,
328,
330,
331,
332,
333,
335,
337,
338,
340,
341,
342,
343,
344,
350,
351,
352,
353,
354,
355,
356,

357,
358,
359,
360,
363,
364,
366,
368,
369,
370,
371,
372,
373,
374,
375,
377,
378,
379,
380,
381,
383,
384,
385,
386,
388,
390,
391,
392,
394,
395,
396,
397,
399,
400,
402,
403,
404,
406,
407,
409,
411,
412,
413,
415,
416,
417,
418,

419,
420,
421,
425,
428,
429,
430,
431,
432,
434,
435,
436,
437,
438,
439,
440,
441,
442,
443,
444,
448,
449,
457,
460,
476,
485,
491,
494,
503,
521,
546,
547,
548,
553,
556,
557,
580,
588,
606,
609,
610,
611,
622,
623,
625,
628,
646,

```
            653,
            660,
            668,
            672}),
 frozenset({1,
            14,
            15,
            19,
            20,
            24,
            25,
            27,
            29,
            31,
            32,
            34,
            35,
            37,
            43,
            48,
            51,
            53,
            55,
            60,
            61,
            76,
            83,
            86,
            88,
            89,
            96,
            101,
            103,
            104,
            105,
            110,
            111,
            112,
            113,
            114,
            116,
            117,
            118,
            119,
            120,
            123,
            124,
```

125,
129,
131,
132,
133,
134,
135,
136,
137,
139,
140,
141,
142,
143,
144,
145,
146,
154,
155,
158,
159,
160,
161,
163,
167,
168,
171,
172,
177,
182,
183,
187,
188,
190,
191,
192,
193,
194,
195,
203,
204,
205,
207,
208,
212,
213,
217,

```
            219,
            223,
            224,
            225,
            226,
            227,
            228,
            234,
            237,
            238,
            242,
            247,
            248,
            251,
            252,
            254,
            255,
            257,
            264,
            267,
            268,
            269,
            271,
            274,
            275,
            276,
            279,
            282,
            283,
            284,
            305,
            326,
            327,
            365,
            376,
            389,
            445,
            450,
            481,
            488,
            501,
            538,
            560,
            564,
            565,
            587}),
frozenset({92,
```

93,
94,
95,
98,
100,
102,
107,
108,
115,
152,
156,
166,
169,
170,
174,
175,
176,
179,
180,
181,
184,
185,
186,
189,
196,
197,
198,
199,
200,
201,
206,
211,
214,
215,
216,
220,
221,
222,
229,
230,
231,
232,
235,
236,
239,
240,
244,

```
            245,
            249,
            250,
            262,
            278,
            280,
            281,
            398}),
  frozenset({97,
            106,
            149,
            164,
            165,
            210,
            218,
            241,
            246,
            256,
            259,
            260,
            261,
            263,
            265,
            266,
            277,
            345,
            361,
            592,
            633,
            669}),
  frozenset({22, 30, 33, 45, 49, 62, 67, 71, 72, 81, 91, 405}),
  frozenset({5, 6, 11, 28, 46, 47, 52, 85}),
  frozenset({463, 509, 608, 612, 630, 667, 675}),
  frozenset({524, 525, 527, 528}),
  frozenset({474, 537, 545}),
  frozenset({334, 339}),
  frozenset({314, 367}),
  frozenset({401, 424}),
  frozenset({473, 651})])
```

[99]:
```
print(f"Task {task_id} - AG tests")
stu_num_community, stu_mod, stu_cov, stu_perf, stu_den, stu_sep,␣
 ↪stu_gred_communities = task_3c_solution(G)

print(f"Task {task_id} - your answer:\n{stu_num_community, stu_mod, stu_cov,␣
 ↪stu_perf, stu_den, stu_sep, stu_gred_communities}")
```

```
assert isinstance(
    stu_num_community, int
), f"Task {task_id}: num_community should be an int. "

assert all(
    isinstance(num, float) for num in [stu_mod, stu_cov, stu_perf, stu_den,␣
 ↪stu_sep]
), (
    f"Task {task_id}: mod, cov, perf, den, and sep should be floats. "
)

assert isinstance(
    stu_lp_communities, list
), f"Task {task_id}: lp_communities should be a list. "

#hidden tests for Question 10 are within this cell
```

```
Task 3b - AG tests
Task 3b - your answer:
(14, 0.5503560299288301, 0.8058922817247199, 0.7750954874009073,
0.6085197486447432, 1.4808561418725692, [frozenset({0, 512, 513, 514, 515, 516,
517, 518, 519, 520, 522, 523, 526, 529, 530, 531, 532, 21, 533, 534, 535, 536,
539, 540, 541, 542, 543, 544, 549, 38, 550, 40, 552, 551, 42, 554, 555, 558,
559, 561, 562, 50, 563, 566, 567, 56, 57, 58, 568, 569, 570, 571, 59, 572, 573,
574, 575, 576, 65, 577, 579, 578, 582, 581, 69, 70, 583, 74, 584, 73, 585, 75,
586, 591, 77, 590, 78, 80, 593, 82, 594, 595, 90, 598, 599, 600, 601, 602, 603,
607, 604, 605, 613, 614, 615, 616, 617, 618, 619, 620, 621, 624, 626, 627, 629,
631, 632, 121, 634, 635, 636, 637, 638, 127, 639, 640, 641, 642, 644, 643, 645,
647, 648, 649, 650, 652, 654, 655, 656, 657, 658, 659, 148, 147, 662, 661, 150,
663, 664, 665, 666, 670, 671, 673, 674, 162, 676, 178, 202, 270, 273, 287, 302,
304, 307, 313, 322, 329, 336, 346, 347, 348, 349, 362, 382, 387, 393, 589, 408,
410, 414, 422, 423, 426, 427, 433, 596, 597, 446, 447, 451, 452, 453, 454, 455,
456, 458, 459, 461, 462, 464, 465, 466, 467, 468, 469, 470, 471, 472, 475, 477,
478, 479, 480, 482, 483, 484, 486, 487, 489, 490, 492, 493, 495, 496, 497, 498,
499, 500, 502, 504, 505, 506, 507, 508, 510, 511}), frozenset({2, 3, 4, 7, 8,
521, 9, 10, 12, 13, 16, 17, 18, 23, 26, 546, 547, 548, 36, 39, 553, 41, 44, 556,
557, 54, 63, 64, 66, 68, 580, 588, 79, 84, 87, 606, 609, 610, 99, 611, 109, 622,
623, 625, 628, 122, 126, 128, 130, 646, 138, 653, 660, 151, 153, 668, 157, 672,
173, 209, 233, 243, 253, 258, 272, 285, 286, 288, 289, 290, 291, 292, 293, 294,
295, 296, 297, 298, 299, 300, 301, 303, 306, 308, 309, 310, 311, 312, 315, 316,
317, 318, 319, 320, 321, 323, 324, 325, 328, 330, 331, 332, 333, 335, 337, 338,
340, 341, 342, 343, 344, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360,
363, 364, 366, 368, 369, 370, 371, 372, 373, 374, 375, 377, 378, 379, 380, 381,
383, 384, 385, 386, 388, 390, 391, 392, 394, 395, 396, 397, 399, 400, 402, 403,
404, 406, 407, 409, 411, 412, 413, 415, 416, 417, 418, 419, 420, 421, 425, 428,
429, 430, 431, 432, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 448,
449, 457, 460, 476, 485, 491, 494, 503}), frozenset({1, 14, 15, 19, 20, 24, 25,
```

```
538, 27, 29, 31, 32, 34, 35, 37, 43, 48, 560, 51, 564, 53, 565, 55, 60, 61, 587,
76, 83, 86, 88, 89, 96, 101, 103, 104, 105, 110, 111, 112, 113, 114, 116, 117,
118, 119, 120, 123, 124, 125, 129, 131, 132, 133, 134, 135, 136, 137, 139, 140,
141, 142, 143, 144, 145, 146, 154, 155, 158, 159, 160, 161, 163, 167, 168, 171,
172, 177, 182, 183, 187, 188, 190, 191, 192, 193, 194, 195, 203, 204, 205, 207,
208, 212, 213, 217, 219, 223, 224, 225, 226, 227, 228, 234, 237, 238, 242, 247,
248, 251, 252, 254, 255, 257, 264, 267, 268, 269, 271, 274, 275, 276, 279, 282,
283, 284, 305, 326, 327, 365, 376, 389, 445, 450, 481, 488, 501}),
frozenset({262, 398, 278, 152, 280, 281, 156, 166, 169, 170, 174, 175, 176, 179,
180, 181, 184, 185, 186, 189, 196, 197, 198, 199, 200, 201, 206, 211, 214, 215,
216, 220, 221, 92, 93, 94, 95, 222, 98, 100, 229, 102, 230, 231, 232, 107, 235,
236, 108, 239, 240, 115, 244, 245, 249, 250}), frozenset({256, 259, 260, 261,
263, 265, 266, 592, 210, 149, 277, 345, 218, 669, 97, 164, 165, 361, 106, 241,
246, 633}), frozenset({33, 67, 71, 72, 45, 81, 49, 405, 22, 62, 91, 30}),
frozenset({52, 5, 6, 85, 11, 28, 46, 47}), frozenset({608, 675, 612, 630, 667,
509, 463}), frozenset({528, 524, 525, 527}), frozenset({537, 474, 545}),
frozenset({339, 334}), frozenset({314, 367}), frozenset({424, 401}),
frozenset({473, 651})])
```

1.5.4 Task 3d:

Using the asynchronous Fluid Communities algorithm with parameters `max_iter` = 100 and `seed` = 233 (see Tutorial for details), find the parameter `k` between `k`=3 and `k`=40 that maximizes the modularity of the partition. Note that `k` represents the number of communities in the partition. Also store the corresponding partition in `fluid_communities`.

[ ]:
```
task_id = '3d'
```

[114]:
```
def task_3d_solution(Graph):

    best_k = None #optimal value of k
    max_mod = 0
    best_k = 0
    fluid_communities= None
    for k in range (3,40):
        com = list(nx.algorithms.community.asyn_fluidc(Graph, k, max_iter=100,␣
 ↪seed=233))
        mod = modularity(Graph, com)
        if mod >= max_mod:
            max_mod, best_k, fluid_communities = mod, k, com

    return best_k, fluid_communities
```

[115]:
```
task_3d_solution(G)
```

[115]:
```
(11,
 [{28,
   46,
```

```
 74,
 90,
 97,
 106,
 108,
 149,
 164,
 165,
 210,
 218,
 241,
 246,
 248,
 249,
 256,
 259,
 260,
 261,
 263,
 265,
 266,
 277,
 313,
 345,
 361,
 382,
 408,
 451,
 501,
 541,
 592,
 633,
 636,
 669},
{17,
 18,
 24,
 39,
 53,
 60,
 76,
 83,
 87,
 292,
 293,
 294,
 295,
```

```
 298,
 299,
 308,
 314,
 316,
 324,
 331,
 332,
 334,
 339,
 343,
 347,
 349,
 357,
 359,
 360,
 365,
 366,
 367,
 369,
 384,
 387,
 402,
 403,
 412,
 416,
 423,
 432,
 476,
 548,
 610,
 623},
{9,
 14,
 20,
 36,
 55,
 68,
 123,
 209,
 243,
 258,
 272,
 286,
 289,
 296,
 300,
```

```
 309,
 312,
 323,
 328,
 330,
 335,
 338,
 344,
 353,
 374,
 376,
 378,
 379,
 381,
 386,
 392,
 395,
 397,
 404,
 418,
 420,
 429,
 434,
 435,
 437,
 440,
 448,
 450,
 611},
{2,
 3,
 4,
 7,
 10,
 12,
 13,
 16,
 26,
 41,
 44,
 66,
 84,
 105,
 109,
 122,
 128,
 130,
```

157,
163,
212,
233,
285,
287,
288,
291,
301,
307,
318,
321,
325,
340,
342,
350,
352,
354,
355,
358,
363,
364,
370,
371,
372,
373,
388,
399,
407,
415,
417,
421,
431,
442,
458,
460,
471,
472,
475,
483,
485,
491,
494,
496,
503,
504,
521,

```
 546,
 547,
 553,
 556,
 557,
 559,
 567,
 571,
 580,
 588,
 606,
 609,
 625,
 628,
 631,
 644,
 648,
 653,
 660,
 668,
 672,
 673},
{15,
 25,
 31,
 32,
 34,
 43,
 61,
 63,
 64,
 86,
 96,
 103,
 104,
 110,
 112,
 113,
 114,
 117,
 119,
 120,
 124,
 125,
 131,
 132,
 133,
```

134,
135,
136,
137,
138,
139,
144,
145,
146,
154,
155,
158,
159,
167,
168,
172,
173,
177,
182,
183,
188,
191,
194,
203,
207,
213,
219,
223,
224,
225,
226,
227,
234,
237,
238,
242,
247,
254,
264,
279,
282,
283,
284,
305,
326,
327,
333,

```
 389,
 445,
 481,
 488},
{56,
 57,
 58,
 59,
 202,
 454,
 455,
 457,
 459,
 463,
 464,
 473,
 498,
 499,
 508,
 509,
 549,
 575,
 581,
 583,
 593,
 595,
 608,
 612,
 614,
 615,
 616,
 617,
 618,
 619,
 622,
 626,
 629,
 630,
 645,
 646,
 647,
 651,
 652,
 658,
 661,
 667,
 675},
```

```
{5,
 6,
 11,
 23,
 38,
 40,
 47,
 52,
 54,
 73,
 79,
 85,
 270,
 290,
 297,
 303,
 304,
 306,
 310,
 311,
 315,
 317,
 319,
 320,
 322,
 329,
 337,
 341,
 348,
 351,
 356,
 368,
 375,
 377,
 380,
 383,
 385,
 390,
 391,
 394,
 396,
 400,
 406,
 409,
 410,
 411,
 413,
```

```
 419,
 422,
 425,
 428,
 430,
 436,
 438,
 439,
 441,
 443,
 444,
 446,
 447,
 449,
 452,
 469,
 510,
 518,
 523,
 552,
 574,
 662,
 663,
 664},
{0,
 19,
 21,
 29,
 35,
 37,
 42,
 50,
 65,
 69,
 70,
 75,
 77,
 80,
 82,
 88,
 148,
 187,
 190,
 192,
 195,
 217,
 255,
```

273,
302,
346,
362,
426,
427,
433,
453,
456,
461,
462,
467,
468,
478,
480,
482,
484,
489,
490,
492,
493,
495,
497,
500,
506,
519,
526,
530,
532,
533,
535,
538,
540,
542,
543,
550,
554,
560,
561,
562,
564,
565,
569,
570,
576,
578,
585,

```
 587,
 590,
 591,
 596,
 599,
 604,
 605,
 613,
 627,
 632,
 634,
 635,
 637,
 638,
 641,
 642,
 649,
 650,
 654,
 655,
 656,
 659,
 666,
 670,
 674},
{22,
 30,
 33,
 45,
 49,
 62,
 67,
 71,
 72,
 81,
 91,
 121,
 147,
 162,
 178,
 336,
 401,
 405,
 424,
 465,
 466,
 474,
```

```
 487,
 502,
 511,
 512,
 513,
 514,
 515,
 516,
 517,
 520,
 522,
 524,
 525,
 527,
 528,
 531,
 534,
 536,
 537,
 539,
 544,
 545,
 555,
 563,
 566,
 568,
 572,
 573,
 577,
 584,
 586,
 589,
 594,
 597,
 600,
 601,
 602,
 624,
 639,
 640,
 643,
 657,
 671,
 676},
{8,
 92,
 93,
```

94,
95,
98,
99,
100,
102,
107,
115,
126,
151,
152,
153,
156,
161,
166,
169,
170,
171,
174,
175,
176,
179,
180,
181,
184,
185,
186,
189,
196,
197,
198,
199,
200,
201,
206,
208,
211,
214,
215,
216,
220,
221,
222,
229,
230,
231,
232,

```
 235,
 236,
 239,
 240,
 244,
 245,
 250,
 253,
 262,
 276,
 278,
 280,
 281,
 398},
{1,
 27,
 48,
 51,
 78,
 89,
 101,
 111,
 116,
 118,
 127,
 129,
 140,
 141,
 142,
 143,
 150,
 160,
 193,
 204,
 205,
 228,
 251,
 252,
 257,
 267,
 268,
 269,
 271,
 274,
 275,
 393,
 414,
```

```
    470,
    477,
    479,
    486,
    505,
    507,
    529,
    551,
    558,
    579,
    582,
    598,
    603,
    607,
    620,
    621,
    665}])
```

[116]:
```
print(f"Task {task_id} - AG tests")
stu_best_k, stu_fluid_communities = task_3d_solution(G)

print(f"Task {task_id} - your answer:\n{stu_best_k, stu_fluid_communities}")

assert isinstance(
    stu_best_k, int
), f"Task {task_id}: best_k should be an int. "

assert isinstance(
    stu_fluid_communities, list
), f"Task {task_id}: fluid_communities should be a list. "

#hidden tests for Question 11 are within this cell
```

```
Task 3b - AG tests
Task 3b - your answer:
(11, [{256, 259, 260, 261, 263, 265, 266, 149, 277, 408, 28, 541, 669, 164, 165,
46, 313, 451, 74, 592, 210, 345, 218, 90, 97, 361, 106, 108, 241, 633, 501, 246,
248, 249, 636, 382}, {384, 387, 17, 402, 403, 18, 24, 412, 416, 292, 293, 548,
39, 295, 294, 298, 299, 423, 432, 308, 53, 314, 316, 60, 324, 331, 332, 76, 334,
83, 339, 87, 343, 347, 476, 349, 610, 357, 359, 360, 365, 366, 623, 367, 369},
{258, 386, 392, 9, 395, 397, 14, 272, 20, 404, 286, 289, 418, 36, 420, 296, 300,
429, 434, 435, 437, 309, 55, 312, 440, 448, 450, 323, 68, 328, 330, 335, 209,
338, 344, 353, 611, 123, 243, 374, 376, 378, 379, 381}, {2, 3, 4, 7, 521, 10,
12, 13, 16, 26, 546, 547, 553, 41, 556, 557, 44, 559, 567, 571, 66, 580, 588,
84, 606, 609, 105, 109, 625, 628, 631, 122, 128, 130, 644, 648, 653, 660, 668,
157, 672, 673, 163, 212, 233, 285, 287, 288, 291, 301, 307, 318, 321, 325, 340,
342, 350, 352, 354, 355, 358, 363, 364, 370, 371, 372, 373, 388, 399, 407, 415,
```

```
417, 421, 431, 442, 458, 460, 471, 472, 475, 483, 485, 491, 494, 496, 503, 504},
{131, 132, 389, 133, 135, 136, 137, 264, 139, 138, 134, 15, 144, 145, 146, 279,
25, 282, 283, 284, 155, 158, 31, 32, 159, 34, 154, 167, 168, 43, 172, 173, 177,
305, 182, 183, 188, 445, 61, 191, 64, 63, 194, 326, 327, 203, 333, 207, 213, 86,
247, 219, 223, 96, 481, 226, 227, 224, 225, 103, 488, 104, 234, 237, 110, 238,
112, 113, 242, 114, 117, 119, 120, 124, 125, 254}, {645, 646, 647, 651, 652,
658, 661, 667, 675, 549, 56, 57, 58, 59, 575, 581, 454, 455, 583, 457, 202, 459,
463, 464, 593, 595, 473, 608, 612, 614, 615, 616, 617, 618, 619, 622, 626, 499,
498, 629, 630, 508, 509}, {385, 5, 518, 6, 391, 390, 394, 11, 396, 523, 270,
400, 662, 23, 663, 406, 409, 411, 664, 413, 410, 290, 419, 38, 422, 40, 297,
552, 441, 428, 425, 430, 47, 304, 303, 306, 436, 52, 310, 438, 439, 54, 311,
443, 444, 317, 574, 319, 315, 320, 449, 446, 322, 452, 447, 73, 329, 79, 337,
341, 85, 469, 348, 351, 356, 368, 375, 377, 380, 510, 383}, {0, 519, 526, 530,
19, 532, 533, 21, 535, 538, 540, 29, 542, 543, 35, 37, 550, 554, 42, 560, 561,
562, 50, 564, 565, 569, 570, 576, 65, 578, 69, 70, 585, 75, 587, 77, 590, 591,
80, 82, 596, 599, 88, 604, 605, 613, 627, 632, 634, 635, 637, 638, 641, 642,
649, 650, 654, 655, 656, 659, 148, 666, 670, 674, 187, 190, 192, 195, 217, 255,
273, 302, 346, 362, 426, 427, 433, 453, 456, 461, 462, 467, 468, 478, 480, 482,
484, 489, 490, 492, 493, 495, 497, 500, 506}, {640, 513, 514, 515, 516, 517,
643, 512, 520, 522, 511, 524, 525, 527, 528, 657, 401, 147, 531, 405, 22, 534,
536, 537, 539, 30, 671, 544, 545, 162, 33, 676, 424, 555, 45, 49, 178, 563, 566,
568, 572, 573, 62, 577, 67, 71, 584, 72, 586, 589, 336, 465, 594, 81, 466, 597,
600, 601, 602, 474, 91, 487, 624, 502, 121, 639}, {262, 8, 398, 276, 278, 151,
152, 281, 153, 280, 156, 161, 166, 169, 170, 171, 174, 175, 176, 179, 180, 181,
184, 185, 186, 189, 196, 197, 198, 199, 200, 201, 206, 208, 211, 214, 215, 216,
220, 93, 222, 221, 95, 94, 98, 99, 100, 92, 230, 102, 232, 229, 231, 235, 236,
107, 239, 240, 115, 244, 245, 250, 253, 126}, {1, 257, 129, 393, 267, 140, 141,
268, 142, 271, 143, 274, 275, 269, 529, 150, 665, 27, 414, 160, 551, 558, 48,
51, 193, 579, 582, 204, 205, 78, 470, 598, 89, 603, 477, 607, 479, 228, 101,
486, 620, 621, 111, 116, 118, 507, 505, 251, 252, 127}])
```

1.5.5 Task 3e.

Using the asynchronous Fluid Communities algorithm with parameters `max_iter` = 100, `seed` = 233, and the value of `k` you found in the previous question, compute the number of communities in the partition and its modularity, coverage, performance, density, and separability.

Hint(s):

- fluid_communities should be converted into a list as in the tutorial.

[ ]:
```
task_id = '3e'
```

[117]:
```
def task_3e_solution(Graph):
    fluid_communities = list(nx.algorithms.community.asyn_fluidc(Graph, k=11,␣
 ↪max_iter=100, seed=233))
    num_community = len(fluid_communities) # number of communities in the␣
 ↪partition
    mod = modularity(Graph, fluid_communities)
```

```
    cov, perf = nx.algorithms.community.partition_quality(Graph,␣
 ↪fluid_communities)
    den = avg_measure(Graph, fluid_communities, task_2b_solution) # density
    sep = avg_measure(Graph, fluid_communities, task_2a_solution) # separability
    return num_community, mod, cov, perf, den, sep, fluid_communities
```

[118]:
```
# Use this cell to explore your solution.

task_3e_solution(G)
```

[118]:
```
(11,
 0.5908556366160678,
 0.728249194414608,
 0.915127651578055,
 0.2114226288880661,
 1.374655967533157,
 [{28,
   46,
   74,
   90,
   97,
   106,
   108,
   149,
   164,
   165,
   210,
   218,
   241,
   246,
   248,
   249,
   256,
   259,
   260,
   261,
   263,
   265,
   266,
   277,
   313,
   345,
   361,
   382,
   408,
   451,
   501,
```

```
 541,
 592,
 633,
 636,
 669},
{17,
 18,
 24,
 39,
 53,
 60,
 76,
 83,
 87,
 292,
 293,
 294,
 295,
 298,
 299,
 308,
 314,
 316,
 324,
 331,
 332,
 334,
 339,
 343,
 347,
 349,
 357,
 359,
 360,
 365,
 366,
 367,
 369,
 384,
 387,
 402,
 403,
 412,
 416,
 423,
 432,
 476,
```

```
 548,
 610,
 623},
{9,
 14,
 20,
 36,
 55,
 68,
 123,
 209,
 243,
 258,
 272,
 286,
 289,
 296,
 300,
 309,
 312,
 323,
 328,
 330,
 335,
 338,
 344,
 353,
 374,
 376,
 378,
 379,
 381,
 386,
 392,
 395,
 397,
 404,
 418,
 420,
 429,
 434,
 435,
 437,
 440,
 448,
 450,
 611},
```

```
{2,
 3,
 4,
 7,
 10,
 12,
 13,
 16,
 26,
 41,
 44,
 66,
 84,
 105,
 109,
 122,
 128,
 130,
 157,
 163,
 212,
 233,
 285,
 287,
 288,
 291,
 301,
 307,
 318,
 321,
 325,
 340,
 342,
 350,
 352,
 354,
 355,
 358,
 363,
 364,
 370,
 371,
 372,
 373,
 388,
 399,
 407,
```

```
 415,
 417,
 421,
 431,
 442,
 458,
 460,
 471,
 472,
 475,
 483,
 485,
 491,
 494,
 496,
 503,
 504,
 521,
 546,
 547,
 553,
 556,
 557,
 559,
 567,
 571,
 580,
 588,
 606,
 609,
 625,
 628,
 631,
 644,
 648,
 653,
 660,
 668,
 672,
 673},
{15,
 25,
 31,
 32,
 34,
 43,
 61,
```

63,
64,
86,
96,
103,
104,
110,
112,
113,
114,
117,
119,
120,
124,
125,
131,
132,
133,
134,
135,
136,
137,
138,
139,
144,
145,
146,
154,
155,
158,
159,
167,
168,
172,
173,
177,
182,
183,
188,
191,
194,
203,
207,
213,
219,
223,
224,

```
 225,
 226,
 227,
 234,
 237,
 238,
 242,
 247,
 254,
 264,
 279,
 282,
 283,
 284,
 305,
 326,
 327,
 333,
 389,
 445,
 481,
 488},
{56,
 57,
 58,
 59,
 202,
 454,
 455,
 457,
 459,
 463,
 464,
 473,
 498,
 499,
 508,
 509,
 549,
 575,
 581,
 583,
 593,
 595,
 608,
 612,
 614,
```

```
 615,
 616,
 617,
 618,
 619,
 622,
 626,
 629,
 630,
 645,
 646,
 647,
 651,
 652,
 658,
 661,
 667,
 675},
{5,
 6,
 11,
 23,
 38,
 40,
 47,
 52,
 54,
 73,
 79,
 85,
 270,
 290,
 297,
 303,
 304,
 306,
 310,
 311,
 315,
 317,
 319,
 320,
 322,
 329,
 337,
 341,
 348,
```

```
 351,
 356,
 368,
 375,
 377,
 380,
 383,
 385,
 390,
 391,
 394,
 396,
 400,
 406,
 409,
 410,
 411,
 413,
 419,
 422,
 425,
 428,
 430,
 436,
 438,
 439,
 441,
 443,
 444,
 446,
 447,
 449,
 452,
 469,
 510,
 518,
 523,
 552,
 574,
 662,
 663,
 664},
{0,
 19,
 21,
 29,
 35,
```

37,
42,
50,
65,
69,
70,
75,
77,
80,
82,
88,
148,
187,
190,
192,
195,
217,
255,
273,
302,
346,
362,
426,
427,
433,
453,
456,
461,
462,
467,
468,
478,
480,
482,
484,
489,
490,
492,
493,
495,
497,
500,
506,
519,
526,
530,
532,

```
 533,
 535,
 538,
 540,
 542,
 543,
 550,
 554,
 560,
 561,
 562,
 564,
 565,
 569,
 570,
 576,
 578,
 585,
 587,
 590,
 591,
 596,
 599,
 604,
 605,
 613,
 627,
 632,
 634,
 635,
 637,
 638,
 641,
 642,
 649,
 650,
 654,
 655,
 656,
 659,
 666,
 670,
 674},
{22,
 30,
 33,
 45,
```

49,
62,
67,
71,
72,
81,
91,
121,
147,
162,
178,
336,
401,
405,
424,
465,
466,
474,
487,
502,
511,
512,
513,
514,
515,
516,
517,
520,
522,
524,
525,
527,
528,
531,
534,
536,
537,
539,
544,
545,
555,
563,
566,
568,
572,
573,
577,

```
 584,
 586,
 589,
 594,
 597,
 600,
 601,
 602,
 624,
 639,
 640,
 643,
 657,
 671,
 676},
{8,
 92,
 93,
 94,
 95,
 98,
 99,
 100,
 102,
 107,
 115,
 126,
 151,
 152,
 153,
 156,
 161,
 166,
 169,
 170,
 171,
 174,
 175,
 176,
 179,
 180,
 181,
 184,
 185,
 186,
 189,
 196,
```

```
 197,
 198,
 199,
 200,
 201,
 206,
 208,
 211,
 214,
 215,
 216,
 220,
 221,
 222,
 229,
 230,
 231,
 232,
 235,
 236,
 239,
 240,
 244,
 245,
 250,
 253,
 262,
 276,
 278,
 280,
 281,
 398},
{1,
 27,
 48,
 51,
 78,
 89,
 101,
 111,
 116,
 118,
 127,
 129,
 140,
 141,
 142,
```

```
    143,
    150,
    160,
    193,
    204,
    205,
    228,
    251,
    252,
    257,
    267,
    268,
    269,
    271,
    274,
    275,
    393,
    414,
    470,
    477,
    479,
    486,
    505,
    507,
    529,
    551,
    558,
    579,
    582,
    598,
    603,
    607,
    620,
    621,
    665}])
```

[119]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_3e_solution(G)

print(f"Task {task_id} - your answer:\n{stu_ans}")

stu_num_community, stu_mod, stu_cov, stu_perf, stu_den, stu_sep,␣
 ↪stu_fluid_communities = stu_ans

assert isinstance(
    stu_num_community, int
), f"Task {task_id}: num_community should be an int. "
```

```
assert all(
    isinstance(num, float) for num in [stu_mod, stu_cov, stu_perf, stu_den,␣
 ↪stu_sep]
), (
    f"Task {task_id}: mod, cov, perf, den, and sep should be floats. "
)

assert isinstance(
    stu_fluid_communities, list
), f"Task {task_id}: lp_communities should be a list. "

#hidden tests for Question 12 are within this cell
```

```
Task 3b - AG tests
Task 3b - your answer:
(11, 0.5908556366160678, 0.728249194414608, 0.915127651578055,
0.2114226288880661, 1.374655967533157, [{256, 259, 260, 261, 263, 265, 266, 149,
277, 408, 28, 541, 669, 164, 165, 46, 313, 451, 74, 592, 210, 345, 218, 90, 97,
361, 106, 108, 241, 633, 501, 246, 248, 249, 636, 382}, {384, 387, 17, 402, 403,
18, 24, 412, 416, 292, 293, 548, 39, 295, 294, 298, 299, 423, 432, 308, 53, 314,
316, 60, 324, 331, 332, 76, 334, 83, 339, 87, 343, 347, 476, 349, 610, 357, 359,
360, 365, 366, 623, 367, 369}, {258, 386, 392, 9, 395, 397, 14, 272, 20, 404,
286, 289, 418, 36, 420, 296, 300, 429, 434, 435, 437, 309, 55, 312, 440, 448,
450, 323, 68, 328, 330, 335, 209, 338, 344, 353, 611, 123, 243, 374, 376, 378,
379, 381}, {2, 3, 4, 7, 521, 10, 12, 13, 16, 26, 546, 547, 553, 41, 556, 557,
44, 559, 567, 571, 66, 580, 588, 84, 606, 609, 105, 109, 625, 628, 631, 122,
128, 130, 644, 648, 653, 660, 668, 157, 672, 673, 163, 212, 233, 285, 287, 288,
291, 301, 307, 318, 321, 325, 340, 342, 350, 352, 354, 355, 358, 363, 364, 370,
371, 372, 373, 388, 399, 407, 415, 417, 421, 431, 442, 458, 460, 471, 472, 475,
483, 485, 491, 494, 496, 503, 504}, {131, 132, 389, 133, 135, 136, 137, 264,
139, 138, 134, 15, 144, 145, 146, 279, 25, 282, 283, 284, 155, 158, 31, 32, 159,
34, 154, 167, 168, 43, 172, 173, 177, 305, 182, 183, 188, 445, 61, 191, 64, 63,
194, 326, 327, 203, 333, 207, 213, 86, 247, 219, 223, 96, 481, 226, 227, 224,
225, 103, 488, 104, 234, 237, 110, 238, 112, 113, 242, 114, 117, 119, 120, 124,
125, 254}, {645, 646, 647, 651, 652, 658, 661, 667, 675, 549, 56, 57, 58, 59,
575, 581, 454, 455, 583, 457, 202, 459, 463, 464, 593, 595, 473, 608, 612, 614,
615, 616, 617, 618, 619, 622, 626, 499, 498, 629, 630, 508, 509}, {385, 5, 518,
6, 391, 390, 394, 11, 396, 523, 270, 400, 662, 23, 663, 406, 409, 411, 664, 413,
410, 290, 419, 38, 422, 40, 297, 552, 441, 428, 425, 430, 47, 304, 303, 306,
436, 52, 310, 438, 439, 54, 311, 443, 444, 317, 574, 319, 315, 320, 449, 446,
322, 452, 447, 73, 329, 79, 337, 341, 85, 469, 348, 351, 356, 368, 375, 377,
380, 510, 383}, {0, 519, 526, 530, 19, 532, 533, 21, 535, 538, 540, 29, 542,
543, 35, 37, 550, 554, 42, 560, 561, 562, 50, 564, 565, 569, 570, 576, 65, 578,
69, 70, 585, 75, 587, 77, 590, 591, 80, 82, 596, 599, 88, 604, 605, 613, 627,
632, 634, 635, 637, 638, 641, 642, 649, 650, 654, 655, 656, 659, 148, 666, 670,
674, 187, 190, 192, 195, 217, 255, 273, 302, 346, 362, 426, 427, 433, 453, 456,
```

```
461, 462, 467, 468, 478, 480, 482, 484, 489, 490, 492, 493, 495, 497, 500, 506},
{640, 513, 514, 515, 516, 517, 643, 512, 520, 522, 511, 524, 525, 527, 528, 657,
401, 147, 531, 405, 22, 534, 536, 537, 539, 30, 671, 544, 545, 162, 33, 676,
424, 555, 45, 49, 178, 563, 566, 568, 572, 573, 62, 577, 67, 71, 584, 72, 586,
589, 336, 465, 594, 81, 466, 597, 600, 601, 602, 474, 91, 487, 624, 502, 121,
639}, {262, 8, 398, 276, 278, 151, 152, 281, 153, 280, 156, 161, 166, 169, 170,
171, 174, 175, 176, 179, 180, 181, 184, 185, 186, 189, 196, 197, 198, 199, 200,
201, 206, 208, 211, 214, 215, 216, 220, 93, 222, 221, 95, 94, 98, 99, 100, 92,
230, 102, 232, 229, 231, 235, 236, 107, 239, 240, 115, 244, 245, 250, 253, 126},
{1, 257, 129, 393, 267, 140, 141, 268, 142, 271, 143, 274, 275, 269, 529, 150,
665, 27, 414, 160, 551, 558, 48, 51, 193, 579, 582, 204, 205, 78, 470, 598, 89,
603, 477, 607, 479, 228, 101, 486, 620, 621, 111, 116, 118, 507, 505, 251, 252,
127}])
```

2 Part 4: Analysis

Run the code below. It will produce a pandas DataFrame where each row represents a community detection algorithm and each column is a quality measure. This will be relevant for the next question.

[120]:
```
lp_communities = task_3b_solution(G)[6]
gred_communities = task_3c_solution(G)[6]
fluid_communities = task_3d_solution(G)[1]

compare = None # assign your pandas dataframe here
method_name = ["Girvan-Newman", "Greedy modularity maximization", "Fluid␣
 ↪communities", "Label propogation"]
compare = pd.DataFrame(method_name, columns=["Method name"])
methods = [max_mod_community, gred_communities, fluid_communities,␣
 ↪lp_communities]
compare['num_community'] = [len(med) for med in methods]
compare['modularity'] = [modularity(G, med) for med in methods]
compare[['coverage', 'performance']] = [nx.community.partition_quality(G, med)␣
 ↪for med in methods]
compare['density'] = [avg_measure(G, med, task_2b_solution) for med in methods]
compare['separability'] = [avg_measure(G, med, task_2a_solution) for med in␣
 ↪methods]
compare
```

[120]:

	Method name	num_community	modularity	coverage	performance	density	separability
0	Girvan-Newman	35	0.580687	0.815252	0.888706	0.654218	1.079485
1	Greedy modularity maximization	14	0.550356	0.805892			
2	Fluid communities	11	0.590856	0.728249			
3	Label propogation	28	0.584188	0.808808			

```
1      0.775095  0.608520      1.480856
2      0.915128  0.211423      1.374656
3      0.857888  0.618999      1.285533
```

3 Task 4a:

For this task, we will log which community detection method performed best according to the quality measures we considered in this module.

Fill the values in the table_analysis function below with strings containing the corresponding Method name as they appear in the table above. For example, if highest_modularity is the Greedy modularity maximization algorithm, you should assign "Greedy modularity maximization" to highest_modularity.

[121]:
```
task_id = '4a'
```

[122]:
```
def task_4a_solution():
    highest_modularity = 'Fluid communities'
    highest_coverage = 'Girvan-Newman'
    highest_performance = 'Fluid communities'
    highest_density = 'Girvan-Newman'
    highest_separability = 'Greedy modularity maximization'

    # YOUR CODE HERE
    #raise NotImplementedError()

    return highest_modularity, highest_coverage, highest_performance,␣
 ↪highest_density, highest_separability
```

[123]:
```
# Use this cell to explore your solution.

task_4a_solution()
```

[123]:
```
('Fluid communities',
 'Girvan-Newman',
 'Fluid communities',
 'Girvan-Newman',
 'Greedy modularity maximization')
```

[124]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_4a_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert all(
    isinstance(num, str) for num in stu_ans
), (
```

```
    f"Task {task_id}: You should return a tuple of strs. "
)

#hidden tests for Question 13 are within this cell
```

```
Task 4a - AG tests
Task 4a - your answer:
('Fluid communities', 'Girvan-Newman', 'Fluid communities', 'Girvan-Newman',
'Greedy modularity maximization')
```

4 Task 4b: Comparing the results with a different network

Now, we will examine a network of football games similar to the one we considered in module 1, with undirected edges forming between teams that played each other at least once in 2000.

You may notice each node has a 'value' field. This refers to the teams' conference.

We will create a table similar to the one in Task 4a, tracking the quality measures for each community detection method, but now using the football network instead of the Wikipedia network

[125]:
```
G_football = nx.read_gml('assets/football.gml', label='id')

max_mod_community_football = None

with open("assets/answer/max_mod_community2", 'rb') as f:
    max_mod_community_football = pickle.load(f)
```

[126]:
```
(list(G_football.nodes(data=True))[0:5])
```

[126]:
```
[(0,
  {'label': 'BrighamYoung',
   'wins': 6,
   'losses': 6,
   'conference': 'Mountain West'}),
 (1,
  {'label': 'FloridaState',
   'wins': 11,
   'losses': 2,
   'conference': 'Atlantic Coast'}),
 (2, {'label': 'Iowa', 'wins': 3, 'losses': 9, 'conference': 'Big Ten'}),
 (3,
  {'label': 'KansasState',
   'wins': 11,
   'losses': 3,
   'conference': 'Big Twelve'}),
 (4,
  {'label': 'NewMexico',
   'wins': 5,
```

```
 'losses': 7,
 'conference': 'Mountain West'})]
```

[127]:
```
lp_communities_football = task_3b_solution(G_football)[6]
gred_communities_football = task_3c_solution(G_football)[6]
fluid_communities_football = task_3d_solution(G_football)[1]

compare = None # assign your pandas dataframe here
method_name = ["Girvan-Newman", "Greedy modularity maximization", "Fluid␣
 ↪communities", "Label propagation"]
compare = pd.DataFrame(method_name, columns=["Method name"])
methods = [max_mod_community_football, gred_communities_football,␣
 ↪fluid_communities_football, lp_communities_football]
compare['num_community'] = [len(med) for med in methods]
compare['modularity'] = [modularity(G_football, med) for med in methods]
compare[['coverage', 'performance']] = [nx.community.
 ↪partition_quality(G_football, med) for med in methods]
compare['density'] = [avg_measure(G_football, med, task_2b_solution) for med in␣
 ↪methods]
compare['separability'] = [avg_measure(G_football, med, task_2a_solution) for␣
 ↪med in methods]
compare
```

[127]:

	Method name	num_community	modularity	coverage	performance	density	separability
0	Girvan-Newman	10	0.599629	0.709625	0.936995	0.769450	1.196155
1	Greedy modularity maximization	6	0.549741	0.730832	0.868192	0.480680	1.341723
2	Fluid communities	13	0.581111	0.663948	0.957437	0.876296	0.984877
3	Label propagation	11	0.583122	0.681892	0.942334	0.813043	1.062906

Fill in the solution strings below as you did in task 4a.

Reflection: Have any answers changed as a result of using a different network? What does this say about the consistency of the community detection methods and the quality measures.

[128]:
```
task_id = '4b'
```

[131]:
```
def task_4b_solution():
    highest_modularity = 'Girvan-Newman'
    highest_coverage = 'Greedy modularity maximization'
    highest_performance = 'Fluid communities'
    highest_density = 'Fluid communities'
```

```
    highest_separability = 'Greedy modularity maximization'

    return highest_modularity,highest_coverage, highest_performance,␣
 ↪highest_density, highest_separability
```

[132]:
```
# Use this cell to explore your solution.

task_4b_solution()
```

[132]:
```
('Girvan-Newman',
 'Greedy modularity maximization',
 'Fluid communities',
 'Fluid communities',
 'Greedy modularity maximization')
```

[133]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_4b_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert all(
    isinstance(num, str) for num in stu_ans
), (
    f"Task {task_id}: You should return a tuple of strs. "
)

#hidden tests for Question 13 are within this cell
```

```
Task 4b - AG tests
Task 4b - your answer:
('Girvan-Newman', 'Greedy modularity maximization', 'Fluid communities', 'Fluid
communities', 'Greedy modularity maximization')
```

5 Task 4c: Community analysis:

Using the method best aligned with modularity according to the table shown at the beginning of part 4, which nodes are in the community with 'NewMexico'? The function below should return a list of integers, where each integer is the id of a node representing a school included in the community with 'NewMexico.' 'NewMexico' itself should appear in the list.

Hint(s): - 'NewMexico' has a node id of 4. - You can obtain community groupings from your solutions in part 3.

[134]:
```
task_id = '4c'
```

[152]:
```
def task_4c_solution():
```

```
    communities = list(nx.algorithms.community.asyn_fluidc(G_football, k=11,␣
 ↪max_iter=100, seed=233))
    newmexico_community = list(list(filter(lambda x: 4 in x, communities))[0])

    return newmexico_community
```

[153]:
```
# Use this cell to explore your solution.

task_4c_solution()
```

[153]:
```
[0, 68, 4, 7, 104, 9, 8, 41, 108, 77, 78, 111, 16, 51, 21, 22, 23, 93]
```

[154]:
```
stu_ans = task_4c_solution()

assert isinstance(
    stu_ans, list
), f"Your function should return a list. "

assert isinstance(
    stu_ans[0], int
), f"Your function should return a list of ints. "
```

Reflection: Below, we print out the values and names of the schools we found in the previous question. As you may recall, the 'conference' value represents the conference the school belongs to. We'd expect schools in the same conference to play each other and therefore be in the same community. Is that what happened here? 7 and 8 correspond to the Mountain West and Pacific Ten conferences, respectively.

[155]:
```
def node_data():
    id_to_value = nx.get_node_attributes(G_football, 'conference')
    id_to_label = nx.get_node_attributes(G_football, 'label')


    schools = task_4c_solution()
    # This section of code converts communities from a list of sets containing␣
 ↪node numbers
    # into a list of sets containing the corresponding college names.
    data = [(id_to_label[node_id], id_to_value[node_id]) for node_id in schools]


    return data


node_data()
```

[155]:
```
[('BrighamYoung', 'Mountain West'),
 ('Oregon', 'Pacific Ten'),
 ('NewMexico', 'Mountain West'),
 ('SouthernCalifornia', 'Pacific Ten'),
 ('NevadaLasVegas', 'Mountain West'),
 ('SanDiegoState', 'Mountain West'),
 ('ArizonaState', 'Pacific Ten'),
 ('ColoradoState', 'Mountain West'),
 ('OregonState', 'Pacific Ten'),
 ('Stanford', 'Pacific Ten'),
 ('WashingtonState', 'Pacific Ten'),
 ('California', 'Pacific Ten'),
 ('Wyoming', 'Mountain West'),
 ('Washington', 'Pacific Ten'),
 ('UCLA', 'Pacific Ten'),
 ('Arizona', 'Pacific Ten'),
 ('Utah', 'Mountain West'),
 ('AirForce', 'Mountain West')]
```

[ ]: